

Porygon: Learning Normal Behaviour Inside Running Containers

Does conditioning a process-execution behavioural baseline on deployment context, in addition to image digest, reduce the false-positive rate without degrading recall?

23CSE498 – Project Phase 2 (Panel Review 1) | Team Number: [XX]

S.No	Register Number	Student Name
1	CB.EN.U4CSE23XXX	Anurup R Krishnan
2	CB.EN.U4CSE23XXX	Student Name 2
3	CB.EN.U4CSE23XXX	Student Name 3
4	CB.EN.U4CSE23XXX	Student Name 4

Guide: **Guide Name**, Designation, Department of CSE

September 6, 2026

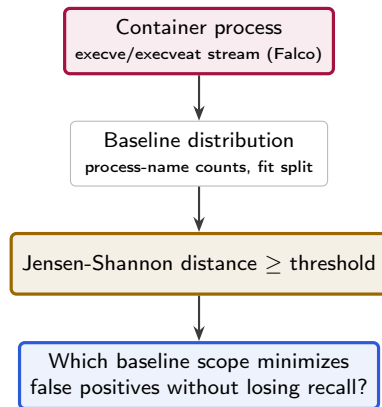
The Problem

Behavioural profile scoping is an unresolved design parameter, not a solved problem

Container process-execution anomaly detection observes the syscalls a containerized process issues (via eBPF/Falco), builds a baseline of “normal” process-name activity, and flags deviating execution events. Falco, Sysdig Secure, Aqua, and academic work since Forrest et al. (1996) already do this; the sensor and scoring primitive are established, not novel.

What every implementation must still decide, without a published answer, is: *at what identity granularity is the baseline scoped?* Per fleet (global), per mutable tag, per exact image digest, or per digest plus runtime security configuration? Vendors pick one by engineering judgement.

The unresolved question. No published, reproducible, statistically powered comparison of these four scoping strategies exists.



A stratum with too few independent fit-split runs returns `insufficient_profile` explicitly, never a guessed score.

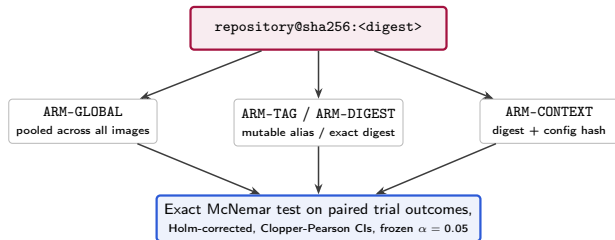
The Research Question

Four scoping arms, one frozen protocol, pre-registered statistics

An OCI image digest (`repository@sha256:<64 hex>`) is an immutable, content-addressed identity: the exact same bytes always resolve to the same digest, unlike a mutable tag such as `nginx:latest`, which can be silently repointed to a different image.

H1 (primary): does scoping the baseline by digest + runtime security context (ARM-CONTEXT) reduce the benign false-positive rate versus digest-only scoping (ARM-DIGEST), without a material loss of detection recall on planted attack scenarios?

Runtime context is a canonical SHA-256 hash over: `entrypoint/command` shape, effective user, privileged, `read_only_rootfs`, network mode, Linux capabilities added/dropped, devices, published ports, and mounts – security-relevant configuration `docker inspect` exposes, excluding secrets and literal argument values.



No fallback between arms during primary analysis. A null result for H1 is reported as a null result, not concealed by a weaker contrast.

What This Is Not, and What It Actually Is

Answering directly: how is this different from Falco, Sysdig, Aqua?

Not novel (used as-is, not claimed as a contribution):

- eBPF/Falco process-execution telemetry
- Behavioural baselining of container process activity
- Jensen-Shannon distance as a categorical divergence measure
- Digest-based image identity
- SBOM + CVE + EPSS + KEV correlation (standard in every CNAPP product)

If asked “**why not just deploy Falco/Sysdig**”: correct answer is that this project does not claim to detect anything Falco cannot already flag with a hand-written rule. It does not compete with those products on detection engineering.

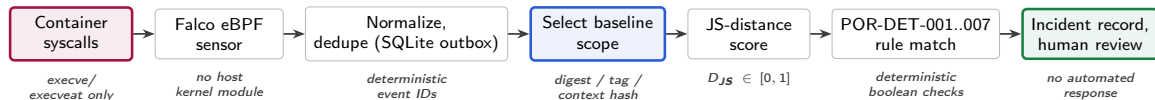
The actual contribution:

- 1 A frozen, pre-registered, falsifiable protocol comparing four baseline-scoping strategies on identical telemetry and rules – vendors pick a scope by judgement; this measures it.
- 2 Exact statistical inference (McNemar, Holm, Clopper-Pearson, bootstrap non-inferiority), full negative result reported.
- 3 One exploratory idea the comparison surfaced: score the runtime security-configuration document directly instead of inferring a configuration change through process-name distance, which the confirmatory data shows cannot see it.

One sentence: a methodology contribution about *how to scope* an existing technique, not a new technique.

1. Methodology — Pipeline

Rubric: Workflow & Algorithmic Pipeline (5 Marks)



Two evidence classes, never merged into one verdict

- **Statistical:** Jensen-Shannon distance against baseline, a rarity measure, not an attack probability
- **Deterministic:** fixed boolean rules (POR-DET-001..007) on shell/tool novelty, root execution, privileged flag, exec activity

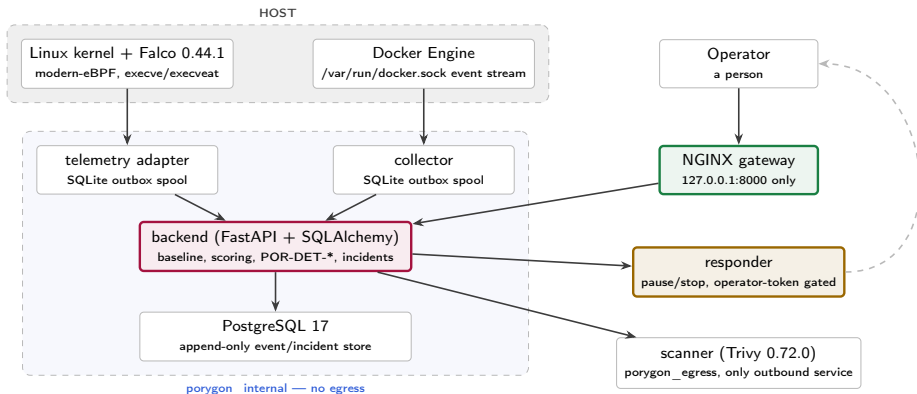
A single merged score would conceal which evidence class fired.

Explicitly out of scope in v1

- No automated containment action without operator approval
- No black-box/opaque model; every score is decomposable per feature
- No network, DNS, socket, or file-access telemetry

1. Methodology — System Architecture

Rubric: Architecture & Trust Boundaries (5 Marks)



Least-privilege per service. Only the collector, scanner, and responder hold Docker-socket-equivalent privilege, for distinct reasons; the backend and telemetry adapter hold neither. Compromise of a Docker-socket service may become host compromise – stated explicitly in docs/THREAT_MODEL_V1.md, not implied away.

1. Methodology — The Scoring Mathematics

Rubric: Mathematical Model / Formulation (5 Marks)

Production path (generates every reported number).

Base-2 Jensen-Shannon distance between baseline p and observed-window q :

$$D_{JS}(p||q) = \sqrt{\frac{1}{2}D_{KL}(p||m) + \frac{1}{2}D_{KL}(q||m)}, \quad m = \frac{p+q}{2}$$

Bounded $[0, 1]$ (base-2 log); 0 = identical. Empty observed window returns None (excluded, not zero); empty baseline vs. non-empty observation returns 1.0.

Aggregation (scoring.py, algorithm porygon.distance.v1, fixed weights, versioned):

$$\text{score} = 0.50 \cdot \overline{D_{JS6 \text{ families}}} + 0.30 \cdot \text{novelty} + 0.20 \cdot \overline{z_5 \text{ numeric}}$$

Categorical weights: process name 0.15, executable 0.25, parent-child 0.20, UID 0.10, runtime action 0.10, sequence bigram 0.20. Missing family excluded, remaining weights renormalized, never imputed as zero.

Study threshold: $D_{JS} > 0.25 = \text{positive call}$

Exploratory alternative, not used for any reported number: calibrated_rarity.py implements unweighted-mean fusion over 4 components and a conformal-style exceedance p-value:

$$p = \frac{1 + \#\{\text{calib. values} \geq \text{test}\}}{n + 1}$$

Feature-flagged (calibrated_enabled, default False); the study used the production JS-distance path, not this module.

Not a probability. A JS-distance or fused rarity value is a distributional-deviation measure. Neither module documents or claims a $P(\text{attack})$ field.

Honest limitation: the 0.50/0.30/0.20 weights are a fixed implementation decision, not fit to data – disclosed, which is why the unweighted alternative exists.

1. Methodology — Dataset, Splits, and Evaluation Metrics

Rubric: Datasets, Preprocessing & Evaluation Metrics (5 Marks)

No public dataset answers this question; data is generated from three real, pinned OCI images (frozen index digest, not a resolvable tag).

Workload	Pinned tag
nginx	1.26.3-alpine
redis	7.2.7-alpine
postgres	16.6-alpine

Two simulated attack scenarios with planted ground truth (SCN-LOG4SHELL-SIM, harmless CVE-2021-44228-shaped spawn; SCN-RUNC-ESCAPE-SIM, harmless CVE-2019-5736-shaped write) plus a capture-integrity canary (SCN-EXEC, not ground truth).

Independent unit: the workload run. Splits assigned per-run by a deterministic trial-ID hash before outcomes are observed.

Primary metrics

Metric ID	Definition
MET-FPR-001	benign runs with ≥ 1 positive call, Clopper-Pearson CI
MET-REC-001	ground-truthed runs detected within horizon
MET-OTP-001	occurrence-to-persistence latency (unmeasured)
MET-CPU-001	CPU-seconds, RSS, disk writes per workload-minute
MET-CAL-001	held-out coverage vs. calibrated threshold

Reporting standard: every percentage ships with numerator/denominator and 95% CI.

2. Implementation Progress — Module Status

Rubric: Core Modules Implemented, Functional & Integrated (8 Marks)

Module	Implementation	Status
Falco eBPF sensor	execve/execveat capture via porygon_rules.yaml	Working
Docker collector	lifecycle events, digest resolution, SQLite outbox spool	Working
Telemetry adapter	Falco JSONL parse, durable cursor, SQLite outbox, retries	Working
Storage & migrations	9 Alembic revisions, append-only tables	Working
Runtime-context hashing	SHA-256 canonical config document (v1 schema)	Working
Baseline builder	per-scope process-name/numeric distributions	Working
JS-distance scorer	scoring.py, versioned config, per-feature breakdown	Working
Deterministic rules	POR-DET-001..007, correlation, allowlists, incidents	Working
Responder	pause/stop, operator-token gated, protected-label guard	Working
Scanner	Trivy 0.72.0 pinned, CycloneDX SBOM, EPSS/KEV correlation	Working
Experiment harness	real-container trials, deterministic split assignment	Working
4-arm scope comparison	144-trial confirmatory run, analysed	Done
Result artifacts	profile-scope-primary.json, incl. null result	Done

Platform: all 8 services healthy end to end. **Evaluation:** n=144 confirmatory round analysed; a 216-trial re-run sized to the recall power requirement ($n_{\text{per-arm}} \approx 179$) is in progress.

2. Implementation Progress — Evidence of Working Software

Rubric: Functional & Integrated (8 Marks)

Codebase

Services (docker compose)	8
Backend API endpoints	64
Alembic migrations	9
Test functions in repo	210

Verification gates, all passing

make verify-static	pass
make verify-unit	pass
make verify-live-safe	pass
Real-container acceptance (scripts/verify_phase*.sh)	pass

Capture-integrity canary, measured (16 runs)

Canary echo events generated	96
Observed at Falco JSONL	96
Rows in PostgreSQL	96
Measured loss	0
Duplicate rows	0
Application requests served	360/360
Orphaned containers after tear-down	0

End-to-end chain, exercised on real data



From a real PostgreSQL container: 479 recorded `execve` events became a baseline, a later window scored `baseline_like`, and 72 POR-DET rule evaluations produced zero false incidents.

2. Implementation Progress — The Safety Rails Work

Refusal paths that prevent an unsupported result from being reported

Three refusal paths that prevent an unsupported or contaminated result from silently becoming a reported number:

1. `insufficient_profile`, not a guessed score.

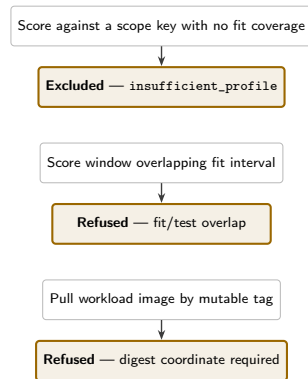
When a scope key (e.g. a digest with zero fit-split coverage) has no reference distribution, `score_trial_under_scope` returns `reference_key_present=False` and the trial is excluded from FPR/recall counts, never scored as either a pass or a detection (`experiments/scope.py`).

2. Fit/test overlap rejection.

Scoring a window that overlaps the interval used to build that same baseline is rejected at the API layer, preventing a model from being evaluated on its own training data.

3. Mutable-reference rejection in the experiment harness.

Pulling a workload image by tag instead of a resolved



Refusals 1 and 3 have named regression tests in `experiments/tests/` (`test_scope.py`, `test_real.py`); refusal 2's exact 409 path is implemented but not yet independently unit-tested – verified here by reading the

source

3. Technical Knowledge — Design Decisions and Rationale

Rubric: Understanding of Methodology, Implementation & Design Choices (5 Marks)

Decision	Rationale	Rejected alternative
Content-addressed digest identity	A mutable tag (nginx:latest) can resolve to a different image tomorrow; a digest cannot without changing identity	Grouping baselines by container name or tag
Kernel-level (eBPF) observation	Captures every <code>execve</code> without application instrumentation or trusting attacker-controlled application logs	In-application logging or agent instrumentation
Reuse Falco as the sensor	Equivalent <code>syscall</code> evidence to a custom probe, far lower implementation/maintenance risk	Writing and maintaining a custom eBPF collector
Fixed, versioned scoring weights	Every historical score stays reproducible under <code>algorithm_version</code> ; changes require a new version, not a silent edit	Weights tuned post hoc until results matched expectation
Exact McNemar/Holm/Clopper-Pearson	A defensible, interval-bounded claim rather than an arbitrary cutoff presented as a rate	Picking a threshold and reporting the resulting accuracy directly
Whole-run split assignment	Adjacent windows in one run are near-duplicates; window-level splitting would overstate confidence	Random assignment of individual time windows
Operator-token-gated containment	A false positive must never autonomously disrupt a healthy service	Automated pause/stop triggered directly by score

3. Technical Knowledge — Trade-offs

Rubric: Design Choices & Trade-offs (5 Marks)

Benefit	Cost
Low-overhead process-execution signal (no app instrumentation)	Zero network, DNS, socket, or file-access telemetry
Finer-grained baselines per digest/context	Fewer independent runs per stratum; some return <code>insufficient_profile</code>
Fixed, disclosed scoring weights	Not claimed to be the most accurate achievable weighting
Exact statistical intervals (Clopper-Pearson, McNemar)	Requires more runs per arm than a point estimate for the same confidence
A fixed, versioned distance-to-score mapping	Only valid while the digest's expected behaviour does not itself drift
No automated disruptive response	Slower operator response time to a confirmed incident
Single language (Python) across all services	A known single-process write-throughput ceiling, not yet exercised as a bottleneck

Edge cases handled in code, with tests

- Backend outage → SQLite outbox spool, exponential backoff retry
- Sensor restart → durable telemetry cursor, no rewind
- Duplicate delivery → deterministic event ID, upsert
- Malformed input → kept as a short, scrubbed sample
- Too little data → answer "insufficient", never guess
- Runtime-context fingerprint redacts secret-shaped flags/values; raw process `command_line` is stored as-is, not redacted

3. Technical Knowledge — Confirmatory Result (n=144)

Rubric: Preliminary Observations (5 Marks) — measured on 144 real container trials

1. ARM-CONTEXT vs ARM-GLOBAL: **significant**.

ARM-GLOBAL (nginx/redis/postgres pooled into one reference) produced a positive call on 35/35 held-out benign runs (FPR 100%, 95% CI 90.0–100%).

Per-image scoping (ARM-CONTEXT) reduced this to 0/35 (FPR 0%, CI 0–10.0%). Exact McNemar $p = 5.8 \times 10^{-11}$, Holm-adjusted, recall non-inferior (62/62 both arms). This is the frozen protocol's H1 contrast; it tests pooling vs. per-image scoping, not context specifically.

2. ARM-CONTEXT vs ARM-DIGEST: **null**.

The paper's actual thesis – does runtime-context conditioning add anything beyond exact digest identity – returns $p = 1.0$: zero discordant pairs, both arms agree on all 35 held-out trials. ARM-CONTEXT beating ARM-GLOBAL above must not be cited as support for this claim; the contrast that actually tests it is a null result.

3. Root cause of the null result.

The tested context variant (dropping Linux capability NET_RAW) changes container privilege, not which processes execute. The categorical process_name feature cannot reflect a capability-only change by construction, independent of scope.

4. Exploratory follow-on, same dataset, no new trials.

context_delta.py scores the runtime-context document directly (privileged, capabilities, read_only_rootfs, network_mode, mounts, devices) instead of inferring it through process names: **0/18 FPR [CI 0–18.5%], 17/17 detection of dropped_capabilities [CI 80.5–100%]**. Hand-fixed weights, not fit; not evaluated against attack scenarios; not confirmatory.

Why this matters: a mechanistically understood null beats an unexplained positive.

3. Technical Knowledge — Claim Boundaries

docs/CLAIMS_V1.md governs exactly what may and may not be stated

Structurally undetectable in v1

- No network, DNS, socket, or file-access telemetry: a trusted binary (e.g. curl) redirected to a malicious destination via config/DNS change produces zero observable deviation
- No new-process signal $\Rightarrow$ invisible (in-memory execution, bash's /dev/tcp)

Full 18-scenario classification in

docs/ADVERSARIAL_SCENARIOS_V1.md: 6/18 structurally invisible (network/data-plane), 4/18 depend on exact process-identity granularity, 3/18 acknowledged but untested, 5/18 genuinely caught by an existing rule.

Honest process note

- 1 First draft framed the easy win as if it supported the harder claim; caught and corrected in self-review before publication.
- 2 Self-reviewed only, not independent peer review.

What is, and is not, yet claimed

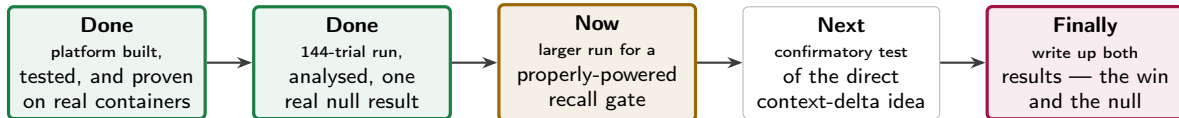
Claim	Status
Per-image beats shared baseline	Supported , $p=5.8 \times 10^{-11}$
Context beats digest-only	Null , $p=1.0$
Context-delta feature works	Exploratory only
Recall properly powered	re-run in progress ($n \approx 179/\text{arm}$)
Suitable for production	Not claimed

Unusual is not the same as malicious.

- Unusual but harmless: backups, debugging, restarts, a traffic spike
- Harmful but ordinary-looking: an attacker using tools already present

Where This Goes Next

From a working platform to an answered, honestly reported question



The intended contribution is not container monitoring itself, which is well-established work. It is a controlled, repeatable comparison of *four different definitions of what counts as the same thing*, reported faithfully whichever way the result falls — including the one comparison that came back null, and the one new idea it led to.

Every figure is traceable to the run that produced it, and every run is retained so that a reviewer can recompute the tables independently.

4. Technical Enrichment Activity Choices

Professional Online Certification Details (Individual Mandatory: 10–20 Hours)

- Individual MOOC / Online Certification Selection:

S.No	Student Name	Chosen MOOC / Course Title	Platform	Hours
1	Anurup R Krishnan	[e.g. Container Security / Linux Observability]	Coursera	15 Hrs
2	Student Name 2	[e.g. Applied Statistics for Experiments]	NPTEL	20 Hrs
3	Student Name 3	[e.g. Docker & Cloud-Native Fundamentals]	edX / Coursera	12 Hrs
4	Student Name 4	[e.g. Cloud Security Foundations]	AWS / Google	16 Hrs

- Requirement:** Each student must complete one approved 10–20 hour online certification from a recognized platform (Coursera, NPTEL, edX, etc.) relevant to the project.
- Note:** course titles above are suggestions aligned to the project's four skill areas — container internals, operating-system observation, experiment design, and security practice. Replace with the actual enrolled courses and attach completion certificates.



S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, "A sense of self for Unix processes," in *Proc. IEEE Symp. Security and Privacy*, 1996, pp. 120–128.



S. A. Hofmeyr, S. Forrest, and A. Somayaji, "Intrusion detection using sequences of system calls," *Journal of Computer Security*, vol. 6, no. 3, pp. 151–180, 1998.



A. S. Abed, T. C. Clancy, and D. S. Levy, "Applying bag of system calls for anomalous behavior detection of applications in Linux containers," in *Proc. IEEE Globecom Workshops*, 2015, pp. 1–5.



J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Trans. Information Theory*, vol. 37, no. 1, pp. 145–151, 1991.



V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY, USA: Springer, 2005.



G. Shafer and V. Vovk, "A tutorial on conformal prediction," *Journal of Machine Learning Research*, vol. 9, pp. 371–421, 2008.



B. Gregg, *BPF Performance Tools: Linux System and Application Observability*. Boston, MA, USA: Addison-Wesley, 2019.



The Falco Authors, "Falco: cloud-native runtime security," Cloud Native Computing Foundation. [Online]. Available: <https://falco.org>



Open Container Initiative, "OCI image format specification." [Online]. Available: <https://github.com/opencontainers/image-spec>

Thank You

Questions welcome, including those on the limits of what this system can currently demonstrate.